

MI IPU API

Version 3.0

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	• Initial release	07/23/2021
	• Added PROCFS introduction	08/25/2021
	• Modify default ipu firmware file path	11/30/2021
	• Modify parameter(pReadCtx) description of MI_IPU_CreateCHN	
	• Modify maximum channel number	
	• Modify the usage of tensor shape	
	• Modify maximum tensor dimension	
	• Modify error codes	
	• Modify ipu frequency path on /sys	
	• Add swdisp related error codes	12/21/2021
	• Add u64BandWidthRead/u64BandWidthWrite to MI_IPU_RuntimeInfo_t structure	
	• Add ipu buffer alignment error code	
	• Modify IPU buffer alignment number	01/20/2022
	• Add MI_IPU_Invoke2Custom function	02/25/2022
	• Modify error codes	
	• Add MI_IPU_FORMAT_GRAY tensor format	
	• Add 'u32BufSize', 'u32InputWidthAlignment', 'u32InputHeightAlignment' and 'bOutputNCHW' in MI_IPU_TensorDesc_t	

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. API 参考.....	1
1.1. 概述.....	1
1.2. 关键词说明.....	1
1.3. 功能模块 API.....	2
1.3.1 MI_IPU_CreateDevice.....	3
1.3.2 MI_IPU_DestroyDevice.....	4
1.3.3 MI_IPU_CreateCHN.....	4
1.3.4 MI_IPU_DestroyCHN.....	6
1.3.5 MI_IPU_GetInOutTensorDesc.....	6
1.3.6 MI_IPU_GetInputTensors.....	8
1.3.7 MI_IPU_PutInputTensors.....	9
1.3.8 MI_IPU_GetOutputTensors.....	10
1.3.9 MI_IPU_PutOutputTensors.....	11
1.3.10 MI_IPU_Invoke.....	12
1.3.11 MI_IPU_GetInputTensors2.....	13
1.3.12 MI_IPU_PutInputTensors2.....	13
1.3.13 MI_IPU_GetOutputTensors2.....	14
1.3.14 MI_IPU_PutOutputTensors2.....	15
1.3.15 MI_IPU_Invoke2.....	16
1.3.16 MI_IPU_Invoke2Custom.....	17
1.3.17 MI_IPU_GetOfflineModeStaticInfo.....	18
2. 数据类型.....	20
2.1. SerializedReadFunc.....	21
2.2. MI_IPU_ELEMENT_FORMAT.....	21
2.3. MI_IPU_TensorDesc_t.....	22
2.4. MI_IPU_SubNet_InputOutputDesc_t.....	23
2.5. MI_IPU_Tensor_t.....	23
2.6. MI_IPU_TensorVector_t.....	24
2.7. MI_IPU_DevAttr_t.....	25
2.8. MI_IPU_ChnAttr_t.....	25
2.9. MI_IPU_BatchInvoke_param_t.....	26
2.10. MI_IPU_RuntimeInfo_t.....	27
2.11. MI_IPU_OfflineModelStaticInfo_t.....	27
3. 错误码.....	29
4. PROCFS 介绍.....	32
4.1. 概述.....	32
4.2. 如何看 MMA 信息.....	32
4.3. 查看 IPU version 信息.....	33
4.4. 查看 IPU clock 信息.....	33
4.5. 调节 IPU clock.....	34
4.6. 开关 auto reset 功能.....	34

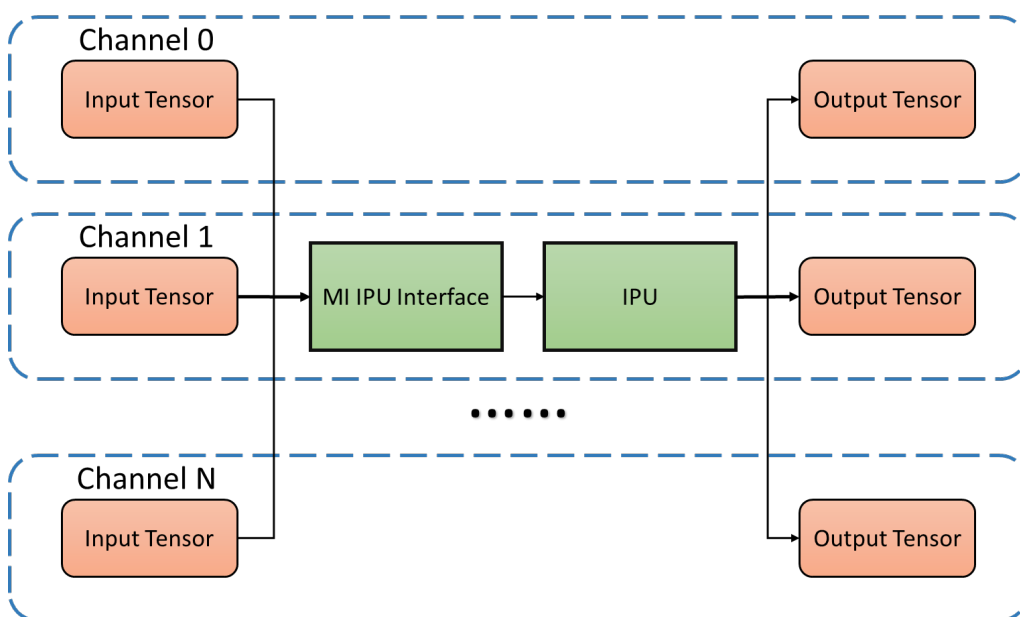
4.7. 抓取 IPU log.....	35
----------------------	----

1. API 参考

1.1. 概述

MI IPU 模块实现了 AI 模型的推演功能加速。通过 channel 支持多个 AI 模型的推演，支持模块内部分配 input/output Tensor，也支持直接使用前一级模块 out buffer 作为 input Tensor。

Padding 框图



1.2. 关键词说明

- IPU
智能处理器
- Firmware
驱动 IPU 硬件的程序文件
- Tensor
网络模型中的多维数据
- Input Tensor
整个 AI 模型的输入 Tensor
- OutputTensor
整个 AI 模型的输出 Tensor

1.3. 功能模块 API

API 名	功能

API 名	功能
MI_IPU_DestroyDevice	销毁 IPU 设备
MI_IPU_CreateCHN	创建 IPU 通道
MI_IPU_DestroyCHN	销毁指定 IPU 通道
MI_IPU_GetInOutTensorDesc	获取指定通道的网络模型输入输出信息
MI_IPU_GetInputTensors	从指定通道分配输入 Tensor Buffer
MI_IPU_PutInputTensors	释放指定通道的输入 Tensor Buffer
MI_IPU_GetOutputTensors	从指定通道分配输出 Tensor Buffer
MI_IPU_PutOutputTensors	释放指定通道的输出 Tensor Buffer
MI_IPU_Invoke	执行一次网络模型推演
MI_IPU_GetInputTensors2	从指定通道分配批处理输入 Tensor Buffer
MI_IPU_PutInputTensors2	释放指定通道的批处理输入 Tensor Buffer
MI_IPU_GetOutputTensors2	从指定通道分配批处理输出 Tensor Buffer
MI_IPU_PutOutputTensors2	释放指定通道的批处理输出 Tensor Buffer
MI_IPU_Invoke2	执行一次 n_buf batch 模式网络模型推演
MI_IPU_Invoke2Custom	执行一次 one_buf batch 模式网络模型推演
MI_IPU_GetOfflineModeStaticInfo	获取离线模型运行需要的 variable buffer size 和离线模型文件大小

1.3.1 MI_IPU_CreateDevice

➤ 功能

创建 IPU 设备。

➤ 语法

```
MI_S32 MI_IPU_CreateDevice(MI_IPU_DevAttr_t *pstIPUDevAttr, SerializedReadFunc pReadFunc,
char *pReadCtx, MI_U32 FWSize);
```

➤ 形参

参数名称	描述	输入/输出
pstIPUDevAttr	IPU 设备属性结构体指针	输入
pReadFunc	用户自定义读取文件函数指针（设为 NULL 默认使用 MI IPU 提供的文件读取函数）	输入
pReadCtx	IPU firmware 文件路径（设为 NULL 默认使用 MI IPU 提供的文件路径：/config/dla/ipu_firmware.bin）	输入
FWSize	IPU firmware 文件大小（设为 0 默认 MI IPU 自动获取文件大小）	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

无。

➤ 举例

```
MI_S32 s32Ret;

stDevAttr.u32MaxVariableBufSize = BufSize; /*模型内部 Tensor 使用的 memory 的最大 size*/
s32Ret = MI_IPU_CreateDevice(&stDevAttr, NULL, NULL, 0);
if (s32Ret != MI_SUCCESS) {
    printf("fail to create ipu device\n");
    return s32Ret;
}
```

➤ 相关主题

无。

1.3.2 MI_IPU_DestroyDevice

- 功能
销毁 IPU 设备。
- 语法
`MI_S32 MI_IPU_DestroyDevice(void);`
- 形参
无
- 返回值
返回值 {
MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。
- 依赖
 - 头文件：mi_ipu.h
 - 库文件：libmi_ipu.so
- ※ 注意
 -
- 举例
`MI_IPU_DestroyDevice();`
- 相关主题
无。

1.3.3 MI_IPU_CreateCHN

- 功能
创建 IPU 通道。
- 语法
`MI_S32 MI_IPU_CreateCHN(MI_IPU_CHN *ptChnId, MI_IPUChnAttr_t *pstIPUChnAttr, SerializedReadFunc pReadFunc, char *pReadCtx);`

- 参数

参数名称	描述	输入/输出
ptChnId	获取 IPU 通道 ID 的指针	输出
pstIPUChnAttr	IPU 通道描述信息结构体指针	输入
pReadFunc	用户自定义读取文件函数（设为 NULL 默认使用 MI IPU 提供	输入

参数名称	描述	输入/输出
	的文件读取函数)	
pReadCtx	网络模型文件路径或者网络模型 OS 内存地址	输入

➤ 返回值

返回值 {
MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

- MI_IPUChnAttr_t.u32BatchMax 为批处理的最大值，如果这个模型不需要使用批处理，则将该值设置为 1 或者 0。
- MI_IPUChnAttr_t.u32InputBufDepth 为设定的输入预分配私有 tensor buffer* u32BatchMax 的个数，比如 0 or 1 or 2 or 3, 当设置为 0 时，表示 buffer 来源于外部。如果直接使用前级模块的输出 MI buffer，建议此处设置 input_depth 为 0 避免内存浪费。
- MI_IPUChnAttr_t.u32OutputBufDepth 为设定的输出预分配私有 tensor buffer* u32BatchMax 的个数，比如 0 or 1 or 2 or 3, 当设置为 0 时，表示 buffer 来源于外部，如 MI_RGN 模块。
- 如果创建的是 one_buf batch 模式网络模型，则不会预分配输入和输出 buffer。
- IPU 最大通道数为 48。

➤ 举例

```
MI_S32 s32Ret, buf_depth = 3, batch_max = 1;
MI_IPU_CHN u32ChnId = 0;
chnAttr.u32InputBufDepth = buf_depth;
chnAttr.u32OutputBufDepth = buf_depth;
chnAttr.u32BatchMax = batch_max;
char pReadCtx[] = "caffe_mobilenet_v2.tflite_sgsimg.img";
s32Ret = MI_IPU_CreateCHN(&u32ChnId, &chnAttr, NULL, pReadCtx);
if (s32Ret != MI_SUCCESS) {
    printf("fail to create ipu channel\n");
    MI_IPU_DestroyDevice();
    return s32Ret;
}
```

- 相关主题
无。

1.3.4 MI_IPU_DestroyCHN

- 功能
销毁指定 IPU 通道。

- 语法
`MI_S32 MI_IPU_DestroyCHN(MI_IPU_CHN u32ChnId);`

- 参数

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入

- 返回值

返回值	MI_SUCCESS 成功。 非 MI_SUCCESS 失败，参照错误码。
-----	---

- 依赖
 - 头文件：mi_ipu.h
 - 库文件：libmi_ipu.so

- ※ 注意
 - IPU 最大通道数为 48。

- 举例
`MI_IPU_DestroyCHN(u32ChnId);`

- 相关主题
无。

1.3.5 MI_IPU_GetInOutTensorDesc

- 描述
获取指定通道的网络模型输入输出信息。

- 语法
`MI_S32 MI_IPU_GetInOutTensorDesc(MI_IPU_CHN u32ChnId, MI_IPU_SubNet_InputOutputDesc_t *pstDesc);`

- 参数

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstDesc	网络描述结构体指针	输出

➤ 返回值

返回值 {
MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

- IPU 最大通道数为 48。

➤ 举例

```
MI_IPU_SubNet_InputOutputDesc_t desc;
s32Ret = MI_IPU_GetInOutTensorDesc(u32ChnId, &desc);
if (s32Ret) {
    printf("fail to get network description\n");
    MI_IPU_DestroyCHN(u32ChnId);
    MI_IPU_DestroyDevice();
    return s32Ret;
}
else {
    int iNum = desc.stMI_InputTensorShapes[0].u32TensorShape[0];
    int iResizeH = desc.stMI_InputTensorShapes[0].u32TensorShape[1];
    int iResizeW = desc.stMI_InputTensorShapes[0].u32TensorShape[2];
    int iResizeC = desc.stMI_InputTensorShapes[0].u32TensorShape[3];
    unsigned char *pu8ImageData = new unsigned char[iNum*iResizeH*iResizeW*iResizeC];
    ...
}
```

➤ 相关主题

无。

1.3.6 MI_IPU_GetInputTensors

➤ 描述

从指定通道获取输入 Tensor Buffer。

➤ 语法

```
MI_S32 MI_IPU_GetInputTensors(MI_IPU_CHN u32ChnId, MI_IPU_TensorVector_t
*pstInputTensorVector);
```

➤ 参数

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInputTensorVector	输入 IPU Tensor 数组结构体指针	输入

➤ 返回值

返回值 {
MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

- 输入 Tensor 的 Buffer 来源有 2 种方式：
 1. 通过 MI_IPU_GetInputTensors 分配 Buffer；
 2. 使用 MI 的其他模块已分配的 Buffer。

➤ 举例

通过 MI_IPU_GetInputTensors 分配 buffer

```
MI_IPU_TensorVector_t inputV;
MI_IPU_TensorVector_t OutputV;
MI_U8 au8MaggiePic[3*224*224] ;
cv::Mat img = cv::imread(filename, -1);
cv::Size inputSize = cv::Size(224,224,3)
cv::Mat imgResize ;
cv::resize(img, imgResize, inputSize);
memcpy(au8MaggiePic, imgResize.data, sizeof(au8MaggiePic));
s32Ret = MI_IPU_GetInputTensors(u32ChnId, &inputV);
if (s32Ret == MI_SUCCESS) {
    memcpy(inputV.stArrayTensors[0].pstTensorData[0], au8MaggiePic, sizeof(au8MaggiePic));
    MI_SYS_FlushInvCache(inputV.stArrayTensors[0].pstTensorData[0], sizeof(au8MaggiePic));
}
else {
    printf("fail to get buffer, please try again");
}
```

```

MI_IPU_GetOutputTensors( u32ChannelID, &OutputV);
if(MI_SUCCESS!=MI_IPU_Invoke(u32ChannelID, & inputV, &OutputV))
{
    cout<<"IPU invoke failed!!"<<endl;

    MI_IPU_DestroyCHN(u32ChannelID);
    MI_IPU_DestroyDevice();
    return -1;
}

```

使用 MI 其他模块已经分配的 buffer

```

MI_IPU_TensorVector_t inputVector, outputVector;
inputVector.u32TensorCount = 1;
if (stBufInfo.eBufType == E_MI_SYS_BUFDATA_RAW) {
    inputVector.astArrayTensors[0].phyTensorAddr[0] = stBufInfo.stRawData.phyAddr;
    inputVector.astArrayTensors[0].ptTensorData[0] = stBufInfo.stRawData.pVirAddr;
} else if (stBufInfo.eBufType == E_MI_SYS_BUFDATA_FRAME) {

    inputVector.astArrayTensors[0].phyTensorAddr[0] = stBufInfo.stFrameData.phyAddr[0];
    inputVector.astArrayTensors[0].ptTensorData[0] = stBufInfo.stFrameData.pVirAddr[0];

    inputVector.astArrayTensors[0].phyTensorAddr[1] = stBufInfo.stFrameData.phyAddr[1];
    inputVector.astArrayTensors[0].ptTensorData[1] = stBufInfo.stFrameData.pVirAddr[1];
}

//prepare output vector
s32Ret = MI_IPU_GetOutputTensors(FdaChn, &outputVector);
s32Ret = MI_IPU_Invoke(FdaChn, &inputVector, &outputVector);

```

- 相关主题
无。

1.3.7 MI_IPU_PutInputTensors

- 功能
释放指定通道的输入 Tensor Buffer。

- 语法
MI_S32 MI_IPU_PutInputTensors(MI_IPU_CHN u32ChnId, MI_IPU_TensorVector_t *pstInputTensorVector);

- 形参

参数名称	描述	输入/输出
------	----	-------

u32ChnId	IPU 通道号	输入
pstInputTensorVector	输入 IPU Tensor 数组结构体指针	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

-

➤ 举例

无。

➤ 相关主题

无。

1.3.8 MI_IPU_GetOutputTensors

➤ 功能

从指定通道获取输出 Tensor Buffer。

➤ 语法

```
MI_S32 MI_IPU_GetOutputTensors(MI_IPU_CHN u32ChnId, MI_IPU_TensorVector_t
*pstInputTensorVector);
```

➤ 形参

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInputTensorVector	输出 IPU Tensor 数组结构体指针	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

无。

➤ 举例

```
MI_IPU_TensorVector_t outputV;
s32Ret = MI_IPU_GetOutputTensors(u32ChnId, &outputV);
if (s32Ret != MI_SUCCESS) {
    printf("fail to get buffer, please try again");
}
```

➤ 相关主题

无。

1.3.9 MI_IPU_PutOutputTensors

➤ 功能

释放指定通道的输出 Tensor Buffer。

➤ 语法

```
MI_S32 MI_IPU_PutOutputTensors(MI_IPU_CHN u32ChnId, MI_IPU_TensorVector_t
*pstInputTensorVector);
```

➤ 形参

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInputTensorVector	输出 IPU Tensor 数组结构体指针	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

1.3.10 MI_IPU_Invoke

➤ 功能

执行一次 AI 网络推演过程。

➤ 语法

```
MI_S32 MI_IPU_Invoke(MI_IPU_CHN u32ChnId, MI_IPU_TensorVector_t *pstInputTensorVector,  
MI_IPU_TensorVector_t *pstOutputTensorVector);
```

➤ 形参

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInputTensorVector	输入 IPU Tensor 数组结构体指针	输入
pstOutputTensorVector	输出 IPU Tensor 数组结构体指针	输入

➤ 返回值

返回值 {
MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

- MI_IPU_Invoke 为同步 API。
- 每一个输入/输出 tensor 的起始物理地址必须 64 bytes 对齐。

➤ 举例

```
s32Ret = MI_IPU_Invoke(u32ChnId, &inputV, &outputV);  
if (s32Ret == MI_SUCCESS) {  
    // process output buffer data  
    // ...  
}
```

➤ 相关主题

无。

1.3.11 MI_IPU_GetInputTensors2

➤ 描述

从指定通道获取批处理输入 Tensor Buffer。

➤ 语法

```
MI_S32 MI_IPU_GetInputTensors2(MI_IPU_CHN u32ChnId, MI_IPU_BatchInvoke_param_t *pstInvokeParam);
```

➤ 参数

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInvokeParam	批处理参数结构体指针	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

输入 Tensor 的 Buffer 来源有 2 种方式：

- 1、通过 MI_IPU_GetInputTensors2 分配 Buffer；
- 2、使用 MI 的其他模块已分配的 Buffer。

➤ 举例

无。

➤ 相关主题

无。

1.3.12 MI_IPU_PutInputTensors2

➤ 功能

释放指定通道的批处理输入 Tensor Buffer。

➤ 语法

```
MI_S32 MI_IPU_PutInputTensors2(MI_IPU_CHN u32ChnId, MI_IPU_BatchInvoke_param_t *pstInvokeParam);
```

➤ 形参

参数名称	描述	输入/输出
------	----	-------

u32ChnId	IPU 通道号	输入
pstInvokeParam	批处理参数结构体指针	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

-

➤ 举例

无。

➤ 相关主题

无。

1.3.13 MI_IPU_GetOutputTensors2

➤ 描述

从指定通道获取批处理输出 Tensor Buffer。

➤ 语法

```
MI_S32 MI_IPU_GetOutputTensors2(MI_IPU_CHN u32ChnId, MI\_IPU\_BatchInvoke\_param\_t
*pstInvokeParam);
```

➤ 参数

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInvokeParam	批处理参数结构体指针	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

1.3.14 MI_IPU_PutOutputTensors2

➤ 功能

释放指定通道的批处理输出 Tensor Buffer。

➤ 语法

MI_S32 MI_IPU_PutOutputTensors2(MI_IPU_CHN u32ChnId, [MI_IPU_BatchInvoke_param_t](#) *pstInvokeParam);

➤ 形参

参数名称	描述	输入/输出
u32ChnId	IPU 通道号	输入
pstInvokeParam	批处理参数结构体指针	输入

➤ 返回值

返回值 { MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

无。

- 举例
无。
- 相关主题
无。

1.3.15 MI_IPU_Invoke2

- 功能
执行一次 n_buf batch 模式网络模型推演。
- 语法
MI_S32 MI_IPU_Invoke2(MI_IPU_CHN u32ChnId, [MI_IPU_BatchInvoke_param_t](#) *pstInvokeParam, [MI_IPU_RuntimeInfo_t](#) *pstRuntimeInfo);

- 形参

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInvokeParam	批处理参数结构体指针	输入
pstRuntimeInfo	IPU 运行信息结构体指针	输入

- 返回值

返回值

{

MI_SUCCESS 成功。

非 MI_SUCCESS 失败，参照[错误码](#)。

- 依赖
 - 头文件：mi_ipu.h
 - 库文件：libmi_ipu.so

※ 注意

- MI_IPU_Invoke2 为同步 API。
- MI_IPU_Invoke2 只能推理 n_buf batch 模式的模型
- 每一个输入/输出 tensor 的起始物理地址必须 64 bytes 对齐。
- 用户指定的 variable buffer 的起始物理地址必须 64 bytes 对齐。

- 举例


```

MI\_IPU\_BatchInvoke\_param\_t stInvokeParam;

MI\_IPU\_RuntimeInfo\_t stRuntimeInfo;

stInvokeParam.u32BatchN = 16;

stInvokeParam.s32TaskPrio = 20;

stInvokeParam.u32IpuAffinity = 0; //由 ipu 调度
      
```

```
s32Ret = MI_IPU_GetInputTensors2(u32ChnId, &stInvokeParam);
if (s32Ret != MI_SUCCESS) {
    printf("Error: MI_IPU_GetInputTensors2 return %d\n", s32Ret);
    return -1;
}
s32Ret = MI_IPU_GetOutputTensors2(u32ChnId, &stInvokeParam);
if (s32Ret != MI_SUCCESS) {
    printf("Error: MI_IPU_GetOutputTensors2 return %d\n", s32Ret);
    MI_IPU_PutInputTensors2(u32ChnId, &stInvokeParam);
    return -1;
}

s32Ret = MI\_IPU\_Invoke2(u32ChnId, &stInvokeParam, &stRuntimeInfo);
if (s32Ret == MI_SUCCESS) {
    // process output buffer data
    // ...

    printf("bw_total=%llu, bw_read=%llu, bw_read=%llu, iputime=%llu\n",
        stRuntimeInfo.u64BandWidth, stRuntimeInfo.u64BandWidthRead,
        stRuntimeInfo.u64BandWidthWrite, stRuntimeInfo.u64IpuTime);
}
```

- 相关主题
无。

1.3.16 MI_IPU_Invoke2Custom

- 功能
执行一次 one_buf batch 模式网络模型推演。

- 语法
MI_S32 MI_IPU_Invoke2Custom(MI_IPU_CHN u32ChnId, [MI_IPU_BatchInvoke_param_t](#) *pstInvokeParam, [MI_IPU_RuntimeInfo_t](#) *pstRuntimeInfo);

- 形参

参数名称	描述	输入/输出
u32ChnId	IPU 通道 ID	输入
pstInvokeParam	批处理参数结构体指针	输入

pstRuntimeInfo	IPU 运行信息结构体指针	输入
----------------	---------------	----

➤ 返回值

返回值 {
MI_SUCCESS 成功。
非 MI_SUCCESS 失败，参照[错误码](#)。

➤ 依赖

- 头文件：mi_ipu.h
- 库文件：libmi_ipu.so

※ 注意

- MI_IPU_Invoke2Custom 为同步 API。
- MI_IPU_Invoke2Custom 只能推理 one_buf batch 模式模型
- 每一个输入/输出 tensor 的起始物理地址必须 64 bytes 对齐。
- 用户指定的 variable buffer 的起始物理地址必须 64 bytes 对齐。

➤ 举例

```
// input tensor number=1 output tensor number=1
MI_IPU_BatchInvoke_param_t stInvokeParam;
MI_IPU_RuntimeInfo_t stRuntimeInfo;
stInvokeParam.u32BatchN = 10;
stInvokeParam.s32TaskPrio = 20;
stInvokeParam.u32IpuAffinity = 0; //由 ipu 调度
int s32InputSize, s32InputSizeOne;
s32InputSizeOne = stDesc.astMI_InputTensorDescs[0].u32BufSize;
s32InputSize = s32InputSizeOne * stInvokeParam.u32BatchN;

int s32OutputSize, s32OutputSizeOne;
s32OutputSizeOne = stDesc.astMI_OutputTensorDescs[0].u32BufSize;
s32OutputSize = s32OutputSizeOne * stInvokeParam.u32BatchN;
s32Ret = MI_SYS_MMA_Alloc(0, NULL, s32InputSize, &u64InputPhyAddr);
if (s32Ret != MI_SUCCESS) {
    printf("fail to allocate input buffer\n");
    return -1;
}
s32Ret = MI_SYS_Mmap(u64InputPhyAddr, s32InputSize, &pInputVirAddr, TRUE);
```

```
if (s32Ret != MI_SUCCESS) {
    MI_SYS_MMA_Free(0, u64InputPhyAddr);
    printf("Error: fail to map input address, error=%d\n", s32Ret);
    return -1;
}

stInvokeParam.astArrayTensors[0].ptTensorData[0] = pInputVirAddr;
stInvokeParam.astArrayTensors[0].phyTensorAddr[0] = u64InputPhyAddr;

s32Ret = MI_SYS_MMA_Alloc(0, NULL, s32OutputSize, &u64OutputPhyAddr);
if (s32Ret != MI_SUCCESS) {
    MI_SYS_Munmap(pInputVirAddr, s32InputSize);
    MI_SYS_MMA_Free(0, u64InputPhyAddr);
    printf("fail to allocate output buffer\n");
    return -1;
}

s32Ret = MI_SYS_Mmap(u64OutputPhyAddr, s32OutputSize, &pOutputVirAddr, TRUE);
if (s32Ret != MI_SUCCESS) {
    MI_SYS_Munmap(pInputVirAddr, s32InputSize);
    MI_SYS_MMA_Free(0, u64InputPhyAddr);
    MI_SYS_MMA_Free(0, u64OutputPhyAddr);
    printf("Error: fail to map output address, error=%d\n", s32Ret);
    return -1;
}

stInvokeParam.astArrayTensors[1].ptTensorData[0] = pOutputVirAddr;
stInvokeParam.astArrayTensors[1].phyTensorAddr[0] = u64OutputPhyAddr;
for (int i = 0; i < stInvokeParam.u32BatchN; i++) {
    memcpy(pInputVirAddr+i*s32InputSizeOne; pInputBuf[i], s32InputSizeOne);
}

MI_SYS_FlushInvCache(pInputVirAddr, s32InputSize);
s32Ret = MI_IPU_Invoke2Custom(channel, &stInvokeParam, NULL);
if (s32Ret != MI_SUCCESS) {
    printf("fail to invoke\n");
}
```



```

MI_SYS_Munmap(pInputVirAddr, s32InputSize);
MI_SYS_MMA_Free(0, u64InputPhyAddr);
MI_SYS_Munmap(pOutputVirAddr, s32OutputSize);
MI_SYS_MMA_Free(0, u64OutputPhyAddr);

return -1;
} else {
    // process output data
    for (int i = 0; i < stInvokeParam.u32BatchN; i++) {
        // pOutputVirAddr+i*s32OutputSizeOne
    }
}

```

- 相关主题
无。

1.3.17 MI_IPU_GetOfflineModeStaticInfo

- 功能
获取离线模型运行需要的 variable buffer size 和离线模型文件大小

- 语法

```
MI_S32 MI_IPU_GetOfflineModeStaticInfo(SerializedReadFunc pReadFunc, char *pReadCtx,
MI_IPU_OfflineModelStaticInfo_t *pStaticInfo);
```

- 形参

参数名称	描述	输入/输出
pReadFunc	用户自定义读取文件函数（设为 NULL 默认使用 MI IPU 提供的文件读取函数	输入
pReadCtx	离线模型文件路径	输入
pStaticInfo	离线模型静态信息结构体指针	输入

- 返回值

返回值

MI_SUCCESS 成功。

非 MI_SUCCESS 失败，参照[错误码](#)。

- 依赖
 - 头文件：mi_ipu.h
 - 库文件：libmi_ipu.so

- ※ 注意

➤ 举例

```
MI_IPU_OfflineModelStaticInfo_t staticInfo;  
s32Ret = MI\_IPU\_GetOfflineModeStaticInfo(NULL, pModelImgPath, &staticInfo);  
if (s32Ret == MI_SUCCESS) {  
    printf("variable buffer size: %u bytes\n%s size: %u bytes\n",  
           staticInfo.u32VariableBufferSize,  
           pModelImgPath,  
           staticInfo.u32OfflineModelSize);  
}
```

➤ 相关主题
无。

2. 数据类型

相关数据类型、数据结构定义如下：

数据类型	定义
SerializedReadFunc	定义用于定制化的读取文件方式，用于支持定制化的 AI 网络存储格式
MI_IPU_ELEMENT_FORMAT	定义 IPU 输入数据枚举类型
MI_IPU_TensorDesc_t	定义 IPU Tensor 形状结构体
MI_IPU_SubNet_InputOutputDesc_t	定义 IPU 子网络输入/输出描述结构体
MI_IPU_Tensor_t	定义 IPU Tensor 地址结构体
MI_IPU_TensorVector_t	定义 IPU Tensor 数组结构体指针
MI_IPU_DevAttr_t	定义 IPU 设备属性结构体
MI_IPU_ChnAttr_t	定义 IPU 通道属性结构体
MI_IPU_BatchInvoke_param_t	定义批处理参数结构体
MI_IPU_RuntimeInfo_t	定义 IPU 运行信息结构体
MI_IPU_OfflineModelStaticInfo_t	定义 IPU 离线模型静态信息结构体

2.1. SerializedReadFunc

➤ 说明

定义用于定制化的读取文件方式，用于支持定制化的 AI 网络存储格式。

➤ 定义

```
typedef int (*SerializedReadFunc)(void *dst_buf,int offset, int size, char *ctx);
```

➤ 成员

成员名称	描述
dst_buf	数据存放地址
offset	文件指针偏移量
size	读取大小
ctx	文件路径

※ 注意事项

无。

➤ 相关数据类型及接口

无。

2.2. MI_IPU_ELEMENT_FORMAT

➤ 说明

定义 IPU 输入数据枚举类型。

➤ 定义

```
typedef enum
{
    MI_IPU_FORMAT_U8,
    MI_IPU_FORMAT_NV12,
    MI_IPU_FORMAT_INT16,
    MI_IPU_FORMAT_INT32,
    MI_IPU_FORMAT_INT8,
    MI_IPU_FORMAT_FP32,
    MI_IPU_FORMAT_UNKNOWN,
    MI_IPU_FORMAT_ARGB8888,
    MI_IPU_FORMAT_ABGR8888,
    MI_IPU_FORMAT_GRAY,
} MI_IPU_ELEMENT_FORMAT;
```

➤ 成员

成员名称	描述
MI_IPU_FORMAT_U8	U8 类型
MI_IPU_FORMAT_NV12	NV12 类型，如 YUV
MI_IPU_FORMAT_INT16	INT16 类型

成员名称	描述
MI_IPU_FORMAT_INT32	INT32 类型
MI_IPU_FORMAT_INT8	INT8 类型
MI_IPU_FORMAT_FP32	FLOAT 类型
MI_IPU_FORMAT_UNKNOWN	Unknown
MI_IPU_FORMAT_ARGB8888	ARGB8888 类型
MI_IPU_FORMAT_ABGR8888	ABGR8888 类型
MI_IPU_FORMAT_GRAY	GRAY 类型

※ 注意事项

ARGB/RGB/BGR tensor 属于 MI_IPU_FORMAT_U8 数据类型
只有 input tensor 可以支援 MI_IPU_FORMAT_NV12 格式
只有 output tensor 可以支援 MI_IPU_FORMAT_FP32 格式

- 相关数据类型及接口
无

2.3. MI_IPU_TensorDesc_t

➤ 说明

定义 IPU Tensor 描述结构体。

➤ 定义

```
typedef struct MI_IPU_TensorDesc_s
{
    MI_U32 u32TensorDim;
    MI_IPU_ELEMENT_FORMAT eElmFormat;
    MI_U32 u32TensorShape[MI_IPU_MAX_TENSOR_DIM];
    MI_S8 name[MAX_TENSOR_NAME_LEN];
    MI_U32 u32InnerMostStride;
    MI_FLOAT fScalar;
    MI_S64 s64ZeroPoint;
    MI_S32 s32AlignedBufSize;
    MI_U32 u32BufSize;
    MI_U32 u32InputWidthAlignment;
    MI_U32 u32InputHeightAlignment;
    MI_BOOL bOutputNCHW;
} MI_IPU_TensorDesc_t;
```

➤ 成员

成员名称	描述
u32TensorDim	Tensor 维度
eElmFormat	Tensor 数据类型
u32TensorShape	Tensor 形状数组
name	Tensor 名称

成员名称	描述
u32InnerMostStride	Tensor 最内维长度（单位字节）
fScalar	Tensor 量化系数
s64ZeroPoint	Tensor 量化 offset
s32AlignedBufSize	Tensor buffer 对齐后的大小
u32BufSize	Tensor buffer 大小
u32InputWidthAlignment	输入 Tensor 水平方向对齐大小
u32InputHeightAlignment	输入 Tensor 垂直方向对齐大小
bOutputNCHW	输出 Tensor 是否是 NCHW 排列

※ 注意事项

- Tensor 维度最大为 10。建议使用如下宏定义：`#define MI_IPU_MAX_TENSOR_DIM (10)`
- 输入数据必须严格按照 `u32InputWidthAlignment` 和 `u32InputHeightAlignment` 对齐，否则结果会出错
- 数据对齐方式

input_formats	板上运行时数据对齐方式
RGB/BGR	不用对齐
RGBA/BGRA	$W = \text{ALIGN_UP}(W * 4, \text{input_width_alignment}) / 4$ <code>input_width_alignment</code> 默认为 1
YUV_NV12	$H = \text{ALIGN_UP}(H, \text{input_height_alignment})$ <code>input_height_alignment</code> 默认为 2 $W = \text{ALIGN_UP}(W, \text{input_width_alignment})$ <code>input_width_alignment</code> 默认为 2
GRAY	$H = \text{ALIGN_UP}(H, \text{input_height_alignment})$ <code>input_height_alignment</code> 默认为 1 $W = \text{ALIGN_UP}(W, \text{input_width_alignment})$ <code>input_width_alignment</code> 默认为 1
RAWDATA_F32_NHWC	不用对齐
RAWDATA_S16_NHWC	不用对齐

- 相关数据类型及接口
无。

2.4. MI_IPU_SubNet_InputOutputDesc_t

- 说明
定义 IPU 子网络输入/输出描述结构体。

- 定义

```
typedef struct MI_IPU_SubNet_InputOutputDesc_s
{
    MI_U32 u32InputTensorCount;
```

```
MI_U32 u32OutputTensorCount;
MI_IPU_TensorDesc_t astMI_InputTensorDescs[MI_IPU_MAX_INPUT_TENSOR_CNT];
MI_IPU_TensorDesc_t astMI_OutputTensorDescs[MI_IPU_MAX_OUTPUT_TENSOR_CNT];
} MI_IPU_SubNet_InputOutputDesc_t;
```

➤ 成员

成员名称	描述
u32InputTensorCount	输入 Tensor 个数
u32OutputTensorCount	输出 Tensor 个数
astMI_InputTensorDescs	输入 Tensor 形状结构体数组
astMI_OutputTensorDescs	输出 Tensor 形状结构体数组

※ 注意事项

无。

➤ 相关数据类型及接口

无。

2.5. MI_IPU_Tensor_t

➤ 说明

定义 IPU Tensor 地址结构体。

➤ 定义

```
typedef struct MI_IPU_Tensor_s
{
    void    *ptTensorData[2];
    MI_PHY  phyTensorAddr[2]; //notice that this is miu bus addr, not cpu bus addr.
} MI_IPU_Tensor_t;
```

➤ 成员

成员名称	描述
ptTensorData	Tensor buffer 虚拟地址
phyTensorAddr	Tensor buffer 物理地址

➤ 注意事项

无。

➤ 相关数据类型及接口

无。

2.6. MI_IPU_TensorVector_t

➤ 说明

定义 IPU Tensor 数组结构体。

➤ 定义

```
typedef struct MI_IPU_TensorVector_s
{
    MI_U32 u32TensorCount;
    MI_IPU_Tensor_t astArrayTensors[MI_IPU_MAX_TENSOR_CNT];
} MI_IPU_TensorVector_t;
```

➤ 成员

成员名称	描述
u32TensorCount	Tensor 个数
astArrayTensors	每个 Tensor 的地址信息

※ 注意事项

无。

➤ 相关数据类型及接口

无。

2.7. MI_IPU_DevAttr_t

➤ 说明

定义 IPU 设备属性结构体。

➤ 定义

```
typedef struct MI_IPU_DevAttr_s
{
    MI_U32 u32MaxVariableBufSize;
    MI_U32 u32YUV420_W_Pitch_Alignment;
    MI_U32 u32YUV420_H_Pitch_Alignment;
    MI_U32 u32XRGB_W_Pitch_Alignment;
} MI_IPU_DevAttr_t;
```

➤ 成员

成员名称	描述
u32MaxVariableBufSize	模型内部 Tensor 使用的 memory 的最大大小
u32YUV420_W_Pitch_Alignment	YUV 水平方向 pitch 大小（网络输入是 YUV 格式时有效）
u32YUV420_H_Pitch_Alignment	YUV 垂直方向 pitch 大小（网络输入是 YUV 格式时有效）
u32XRGB_W_Pitch_Alignment	RGBA/BGRA 水平方向 pitch 大小（网络输入是 RGBA/BGRA 格式时有效）

※ 注意事项

无。

➤ 相关数据类型及接口

无。

2.8. MI_IPU_ChnAttr_t

➤ 说明

定义 IPU 通道属性结构体。

➤ 定义

```
typedef struct MI_IPU_ChnAttr_s
{
    MI_U32 u32SubNetId;
    MI_U32 u32OutputBufDepth;
    MI_U32 u32InputBufDepth;
    MI_U32 u32BatchMax;
} MI_IPUChnAttr_t;
```

➤ 成员

成员名称	描述
u32SubNetId	子网络 ID
u32OutputBufDepth	输出 Tensor Buffer 深度
u32InputBufDepth	输入 Tensor Buffer 深度
u32BatchMax	批处理的最大值

※ 注意事项

IPU 输入/输出缓冲区最大深度为 3。建议使用如下宏定义：

```
#define MAX_IPU_INPUT_OUTPUT_BUF_DEPTH (3)
```

➤ 相关数据类型及接口

无。

2.9. MI_IPU_BatchInvoke_param_t

➤ 说明

定义批处理参数结构体。

➤ 定义

```
typedef struct MI_IPU_BatchInvoke_param_s {  
    MI_PHY u64VarBufPhyAddr;  
    MI_U32 u32VarBufSize;  
    MI_U32 u32BatchN;  
    MI_S32 s32TaskPrio;  
    MI_U32 u32IpuAffinity;  
    MI_IPU_Tensor_t astArrayTensors[MI_IPU_MAX_BATCH_TENSOR_CNT];  
} MI_IPU_BatchInvoke_param_t;
```

➤ 成员

成员名称	描述
u64VarBufPhyAddr	用户指定的 variable buffer 物理地址
u32VarBufSize	用户指定的 variable buffer 大小
u32BatchN	批处理个数
s32TaskPrio	任务优先级
u32IpuAffinity	绑定 ipu core
astArrayTensors	批处理所有的输入输出 tensor buffer 地址

※ 注意事项

- astArrayTensors 数组存放批处理所有的输入输出 tensor buffer 地址，规则是先依次存入所有输入 tensor buffer 地址，再存入所有输出 tensor buffer 地址。
- 每一个输入/输出 tensor 的起始物理地址必须 64 bytes 对齐。
- 用户指定的 variable buffer 的起始物理地址必须 64 bytes 对齐。

- 相关数据类型及接口
无。

2.10. MI_IPU_RuntimeInfo_t

- 说明

定义 IPU 运行信息结构体。

- 定义

```
typedef struct MI_IPU_RuntimeInfo_s {
    MI_U64 u64BandWidth;
    MI_U64 u64IpuTime;
    MI_U64 u64BandWidthRead;
    MI_U64 u64BandWidthWrite;
} MI_IPU_RuntimeInfo_t;
```

- 成员

成员名称	描述
u64BandWidth	带宽数据总量
u64IpuTime	IPU 处理时间
u64BandWidthRead	带宽数据读总量
u64BandWidthWrite	带宽数据写总量

- ※ 注意事项
无

- 相关数据类型及接口
无。

2.11. MI_IPU_OfflineModelStaticInfo_t

- 说明

定义 IPU 离线模型静态信息结构体。

- 定义

```
typedef struct MI_IPU_OfflineModelStaticInfo_s {
    MI_U32 u32VariableBufferSize;
    MI_U32 u32OfflineModelSize;
} MI_IPU_OfflineModelStaticInfo_t;
```

- 成员

成员名称	描述
u32VariableBufferSize	离线模型运行需要的 variable buffer size
u32OfflineModelSize	离线模型文件大小

- ※ 注意事项
无。
- 相关数据类型及接口
无。

3. 错误码

错误代码	宏定义	描述
0	MI_SUCCESS	Success
1	E_IPU_ERR_INVALID_CHNID	Invalid channel ID
2	E_IPU_ERR_CHNID_EXIST	Channel already exists
3	E_IPU_ERR_CHNID_UNEXIST	Channel does not exist
4	E_IPU_ERR_NOMEM	Failure caused by malloc memory
5	E_IPU_ERR_NOBUF	Failure caused by malloc buffer
6	E_IPU_ERR_BADADDR	Bad address, buffer address is not gotten from IPU buffer allocator
7	E_IPU_ERR_SYS_TIMEOUT	System timeout
8	E_IPU_ERR_FILE_OPERATION	File cannot be opened or read or written
9	E_IPU_ERR_ILLEGAL_TENSOR_BUFFER_SIZE	Tensor buffer size does not meet the requirement, usually less than the requirement
10	E_IPU_ERR_ILLEGAL_BUFFER_DEPTH	Input or output buffer depth quantum exceeds maximum number
11	E_IPU_ERR_ILLEGAL_INPUT_OUTPUT_DESC	Network description is illegal, usually input or output buffer quantum is wrong
12	E_IPU_ERR_ILLEGAL_INPUT_OUTPUT_PARAMETER	User's input or output buffer quantum does not match network description
13	E_IPU_ERR_MAP	Address mapping error
14	E_IPU_ERR_INIT_FIRMWARE	Fail to initialize IPU firmware
15	E_IPU_ERR_CREATE_CHANNEL	Fail to create channel
16	E_IPU_ERR_DESTROY_CHANNEL	Fail to destroy channel
17	E_IPU_ERR_INVOKE	Fail to invoke
18	E_IPU_ERR_SET_MALLOC_REGION	Fail to set malloc region for freertos
19	E_IPU_ERR_SET_IPU_PARAMETER	Fail to set IPU parameter
20	E_IPU_ERR_INVALID_PITCH_ALIGNMENT	Invalid pitch alignment
21	E_IPU_ERR_NO_CREATED_IPU_DEVICE	There is no created IPU device
22	E_IPU_ERR_GET_IPU_VERSION	Fail to get IPU version from IPU firmware
23	E_IPU_ERR_MISMATCH_IPU_HEAD_FILE	IPU head files version not matched

错误代码	宏定义	描述
24	E_IPU_ERR_NO_SUPPORT_REQ	IPU firmware does not support this request
25	E_IPU_ERR_FAILED	Unexpected error
26	E_IPU_ERR_SEND_REQUEST	Fail to send request to IPU
27	E_IPU_ERR_GET_FIRMWARE_INFO	Fail to get ipu firmware information
28	E_IPU_ERR_INVALID_IPUCORE_BOOTING_PARAMETERS	Invalid IPU cores booting parameters
29	E_IPU_ERR_INVALID_IPUCORE_SHUTDOWN_PARAMS	Invalid IPU cores shutdowning parameters
30	E_IPU_ERR_NO_MULTICORE_ENV	Multicore mode needs all ipu cores being alive
31	E_IPU_ERR_INVALID_TASK_PRIORITY	Invalid ipu task priority
32	E_IPU_ERR_DEV_SHUTDOWN	Ipu core has been shutdown
33	E_IPU_ERR_DEV_FAIL_RESET	Fail to reset ipu
34	E_IPU_ERR_DEV_FAIL_SHUTDOWN	Fail to shutdown ipu
35	E_IPU_ERR_NO_AVAILABLE_DEV	No available ipu dev
36	E_IPU_ERR_RESET_OFF	Reset function is off
37	E_IPU_ERR_INVALID_BATCH_NUM	batch number error
38	E_IPU_ERR_BATCH_TYPE	batch type error
39	E_IPU_ERR_BATCH_MODE	batch mode error
40	E_IPU_ERR_NO_AVAILABLE_BATCH_MODE	do not find available batch mode
41	E_IPU_ERR_IPU_HANG	invoke was dropped due to ipu hang
42	E_IPU_ERR_NO_RESET_DEV	no reset ipu dev
43	E_IPU_ERR_NO_BATCH_PARAM	no batch parameter
44	E_IPU_ERR_INVALID_MODEL_BUFFER	invalid user model buffer physical address or size
45	E_IPU_ERR_INVALID_VARIABLE_BUFFER	invalid variable buffer physical address or size
46	E_IPU_ERR_NOT_ASSIGN_CORE	not assign ipu core when use user's variable buffer
47	E_IPU_ERR_SWDISP_NOT_REGISTER	model has unsupported swdisp function
48	E_IPU_ERR_SWDISP_NOT_FIND_TASKID	not find swdisp task id
49	E_IPU_ERR_SWDISP_INVALID_PARAM	invalid swdisp parameter
50	E_IPU_ERR_SWDISP_UNEXPECTED	unexpected swdisp error
51	E_IPU_ERR_SWDISP_UNKNOWN	unknown swdisp error
52	E_IPU_ERR_BAD_PHY_ADDR_ALIGNMENT	ipu buffer physical addr not aligned

错误代码	宏定义	描述
53	E_IPU_ERR_MISMATCH_INVOKE_FUNC	n_buf/one_buf batch model should use MI_IPU_Invoke2/MI_IPU_Invoke2Custom
256	E_IPU_ERR_NO_AVAILABLE_CHNID	There is no available channel

4. PROCFS 介绍

4.1. 概述

通过控制台 debug IPU 的方式主要有 procfs 和 sysfs。

- IPU Procfs 在 open device (mi_dev) 的时候创建节点” proc/mi_module/mi_ipu/mi_ipu0”，close device 的时候删除节点。
- IPU Sysfs 在 insmod ko 的时候创建节点” sys/dla/xxx”。

4.2. 如何看 MMA 信息

指令: cat /proc/mi_modules/mi_sys/mi_sys0

```
/ # cat /proc/mi_modules/mi_sys/mi_sys0
```

-----mma allocator info for DevId:0-----									
mma_heap_name0	nameheap_base_cpu_addr	length	chunk_mgr_avail	mmuEnable	addsize_before	addsize_after	bMmuDelayUnmap		
chunk_mgr_info:	offset	length	avail						
	0	10000000	f4e6000						
each chunk info:									
offset	length	used_flag	task_name	pid	Module				
0	c0000	1	CMDMEM	-1	mi_sys				
c0000	1000	1	sys-logConfig	-1	mi_sys				
c1000	40000	1	sys-logBuffer	-1	mi_sys				
101000	1a1000	1	ipu_variable	-1	mi_ipu				
2a2000	400000	1	ipu_heap	-1	mi_ipu				
6a2000	2000	1	IPU log	-1	mi_ipu				
6a4000	407000	1	ipu_chn0_networ	720	mi_ipu				
aab000	a000	1	ipu_chn0_desc	720	mi_ipu				
ab5000	1000	1	IPU BufferChn0	720	mi_ipu				
ab6000	31000	1	ipu_chn0_input_	720	mi_ipu				
ae7000	31000	1	ipu_chn0_input_	720	mi_ipu				
b18000	1000	1	ipu_chn0_output	720	mi_ipu				
b19000	1000	1	ipu_chn0_output	720	mi_ipu				
b1a000	f4e6000	0	ipu_chn0_output	-1	mi_sys				
mma_heap_ipu	2ffe0000	164000	0	0	0	0	0		
chunk_mgr_info:	offset	length	avail						
	0	164000	0						
each chunk info:									
offset	length	used_flag	task_name	pid	Module				
0	164000	1	ipu_firmware	-1	mi_ipu				
imi_mma_heap	20000000	43800	43800	0	0	0	0		
chunk_mgr_info:	offset	length	avail						
	0	43800	43800						
each chunk info:									
offset	length	used_flag	task_name	pid	Module				
0	43800	0	NA	-1	mi_sys				

可以看到所有 MMA 的使用情况。

指令: cat /proc/mi_modules/mi_ipu/mi_ipu0

```
/ # cat /proc/mi_modules/mi_ipu/mi_ipu0
```



```
-----CMDQ Kickoff counter-----
DevId  current_buf_size  Peak_buf_size
0      707000           70b000
each dev buf info:
      offset      length      used_flag      task_name
      101000      1a1000      1              ipu_variable
      2a2000      400000      1              ipu_heap
      6a2000      2000       1              IPU_log
      0           164000      1              ipu_firmware
-----Common info formi_ipu only dump enabled chn-----
ChnId  PassNum  EnInPNum  EnOutPNum  MMAHeapName
0      0        0         0          (null)
ChnId  current_buf_size  Peak_buf_size  user_pid
0      476000          476000         720
each chn buf info:
      offset      length      used_flag      task_name
      6a4000      407000      1              ipu_chn0_networ
      aab000      a000       1              ipu_chn0_desc
      ab5000      1000       1              IPU_Buffer_chn0
      ab6000      31000      1              ipu_chn0_input_
      ae7000      31000      1              ipu_chn0_input_
      b18000      1000       1              ipu_chn0_output
      b19000      1000       1              ipu_chn0_output
```

可以看 IPU 的 device 和各 channel 使用 MMA 的情况。

4.3. 查看 IPU version 信息

指令: `echo version > /proc/mi_modules/mi_ipu/mi_ipu0`

```
/ # echo version > /proc/mi_modules/mi_ipu/mi_ipu0
lib version: P_1.0.1_210525-86ece6aa9a0cfe21fd30126fcb92b183
driver version: P_1.0.1_210525
```

指令: `cat /sys/dla/version`

```
/ # cat /sys/dla/version
lib version: P_1.0.1_210525-86ece6aa9a0cfe21fd30126fcb92b183
driver version: P_1.0.1_210525
```

4.4. 查看 IPU clock 信息

指令: `cat /sys/dla/freq`

```
/ # cat /sys/dla/freq
current ipu clock frequency: 800MHz
available frequency:
1500MHz
1400MHz
1300MHz
1200MHz
1100MHz
1000MHz
900MHz
800MHz
700MHz
600MHz
500MHz
400MHz
300MHz
200MHz
150MHz
```

4.5. 调节 IPU clock

指令: `echo xxx > /sys/dla/freq`

(XXX 必须是 available frequency)

```
/ # echo 900 > /sys/dla/freq
/ # cat /sys/dla/freq
current ipu clock frequency: 900MHz
available frequency:
1500MHz
1400MHz
1300MHz
1200MHz
1100MHz
1000MHz
900MHz
800MHz
700MHz
600MHz
500MHz
400MHz
300MHz
200MHz
150MHz
```

4.6. 开关 auto reset 功能

Auto reset 功能可以在 IPU 无响应后 reset IPU，继续未完成的任务。

打开 auto reset 功能: `echo on > /sys/dla/auto_reset`

关闭 auto reset 功能: `echo off > /sys/dla/auto_reset`

前期测试阶段，建议关闭 auto reset 功能，理清 IPU 无响应的原因。

4.7. 抓取 IPU log

指令:

```
echo "ctrl_size=0x800000 corectrl_size=0x800000 ctrl=0xffffffff corectrl=0x1fff" > /sys/dla/ipu_log
```

ctrl_size 和 corectrl_size 是为 ctrl log 和 corectrl log 分配 buffer 的大小，ctrl 和 corectrl 是 ctrl log 和 corectrl log 的配置。